

AI201: Assignment 1 Solutions

Roll Number: 2025390

A1

$$J(w, b) = \frac{1}{4} \sum_{i=1}^4 (wx_i + b - y_i)^2$$

x_i, y_i belong to $\{(-1, -1), (0, 1), (1, 2), (2, 5)\}$

A

DERIVE $j(w, b)$

$$\frac{\partial J}{\partial w} = \frac{\partial \left(\frac{1}{4} \sum_{i=1}^4 (wx_i + b - y_i)^2 \right)}{\partial w} = \frac{2}{4} \sum_{i=1}^4 x_i (wx_i + b - y_i)$$

$$\frac{\partial J}{\partial b} = \frac{1}{2} \sum_{i=1}^4 (wx_i + b - y_i) = \frac{1}{2} [w \sum_{i=1}^4 x_i + 4b - \sum y_i]$$

$$\sum x_i = 2, \quad \sum x_i^2 = 6 \quad \sum y_i = 7, \quad \sum x_i y_i = 13$$

$$\frac{\partial J}{\partial w} = \frac{1}{2} (6w + 2b - 13)$$

$$\frac{\partial J}{\partial b} = \frac{1}{2} (2w + 4b - 7)$$

$$j(w, b) = [3w + b - 6.5w + 2b - 3.5]$$

For hessian we need to differentiate the gradient so

$$Hj(w, b) = \begin{bmatrix} \frac{\partial^2 J}{\partial w^2} & \frac{\partial^2 J}{\partial w \partial b} \\ \frac{\partial^2 J}{\partial b \partial w} & \frac{\partial^2 J}{\partial b^2} \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\det(Hj) = (3)(2) - (1)(1) = 5 > 0 \text{ and } \frac{\partial^2 J}{\partial w^2} > 0$$

its strictly convex as its convex hence it has only 1 point as global minimum.

$$(w, b) = (0, 0),$$

$$\eta = 0.1$$

$$w = w - \eta \frac{\partial J}{\partial w}, \quad b = b - \eta \frac{\partial J}{\partial b}$$

Initially $w=b=0$ hence $wx_i+b=0$; hence

$$J(0, 0) = MSE = \frac{1}{4} \sum_{i=1}^4 (0 - y_i)^2$$

$$J(0, 0) = \frac{1}{4} \sum_{i=1}^4 (0 - y_i)^2 = \frac{1}{4} (1 + 1 + 4 + 25) = \frac{31}{4} = 7.75$$

So our gradient vector became like

$$\nabla J(0,0) = [3 * 0 + 0 - 6.5, 0 + 2 * 0 - 3.5] = [-6.5 \quad -3.5]$$

And parameter

$$[w_1, b_1] = [w_0, b_0] - \eta \nabla J(0,0) = [0 \ 0] - 0.1[-6.5 \quad -3.5] = [0.65 \ 0.35]$$

$$w_1 = 0.65, \quad b_1 = 0.35$$

$$\hat{y}_i = 0.65x_i + 0.35:$$

$$\hat{y}_1 = 0.65(-1) + 0.35 = -0.3 \Rightarrow y_1 - \hat{y}_1 = -1 - (-0.3) = -0.70^2 = 0.49$$

$$\hat{y}_2 = 0.65(0) + 0.35 = 0.35 \Rightarrow y_2 - \hat{y}_2 = 1 - 0.35 = 0.65^2 = 0.4225$$

$$\hat{y}_3 = 0.65(1) + 0.35 = 1.00 \Rightarrow y_3 - \hat{y}_3 = 2 - 1.00 = 1.00^2 = 1.00$$

$$\hat{y}_4 = 0.65(2) + 0.35 = 1.65 \Rightarrow y_4 - \hat{y}_4 = 5 - 1.65 = 3.35^2 = 11.2225$$

So mse becomes

$$J(0.65, 0.35) = \frac{1}{4} [0.49 + 0.4225 + 1.00 + 11.2225] = \frac{13.135}{4} = 3.28375$$

c.

Incorrect: `dw = np.mean(error), np.mean(error * x)`

1. In part (a) $\frac{\partial J}{\partial w} = \frac{1}{n} \sum (wx_i + b - y_i)x_i$, so `dw = np.mean(error)` is incorrect as there is no x multiplied to it. The student swapped x between `dw` and `db`. It should be `np.mean(error * x)` for `dw` and `np.mean(error)` for `db`.
2. Also `w = w + lr * dw` and `b = b + lr * db` is wrong because gradient descent has subtraction.

Corrected:

```
pred = w * x + b
error = pred - y
dw = np.mean(error * x)
db = np.mean(error)
w = w - lr * dw
b = b - lr * db
```

d.

$$g(u, v) = [u^2 v u + v^3]:$$

$$J_g(u, v) = \left[\frac{\partial g_1}{\partial u} \quad \frac{\partial g_1}{\partial v}, \quad \frac{\partial g_2}{\partial u} \quad \frac{\partial g_2}{\partial v} \right] = [2uv u^2, 1 \ 3v^2]$$

at $(u, v) = (1, 2)$:

$$J_g(1, 2) = \begin{bmatrix} 2(1)(2) & 1^2 & 1 & 3(2^2) \end{bmatrix} = [4 \ 1 \ 1 \ 12]$$

The Jacobian represents the first order partial derivative of a vector function to describe local sensitivity or linear transformation. While Hessian in part is a second order derivative matrix of a scalar function which describes the structure curvature convex, concave, etc

(e)

Minimizing empirical loss fits a model specifically to training data. It can give low loss on training samples, but for unseen future data it doesn't guarantee small expected risk because future observations might come from a slightly different sampling variation or underlying distribution, leading to high expected loss due to overfitting.

A2

A.

$$P(D = 1) = 0.02, P(D = 0) = 0.98$$

$$P(T = 1|D = 1) = 0.95, \quad P(T = 1|D = 0) = 0.10$$

So we need $p(T=1)$ AS

$$P(D = 1|T = 1) = \frac{P(T=1|D=1)P(D=1)}{P(T=1)}$$

$$P(T = 1) = P(T = 1|D = 1)P(D = 1) + P(T = 1|D = 0)P(D = 0) \\ = 0.95 \times 0.02 + 0.10 \times 0.98 = 0.019 + 0.098 = 0.117$$

Applying Bayes rule for posterior:

$$P(D = 1|T = 1) = \frac{P(T=1|D=1)P(D=1)}{P(T=1)} = \frac{0.019}{0.117} = \frac{19}{117} \approx 0.1624$$

The manager process miscalculates the base rate by equating sensitivity

$P(T = 1|D = 1)$ directly to posterior, ignoring $P(D = 1) = 0.02$. Since true defects are very rare, false positives (0.098) end up outnumbering true positives (0.019). Hence posterior is far from 0.95. It also depends on $P(D=1)$ hence it is far.

B.

$$x_1 = [2 \ 3], \quad x_2 = [4 \ 7], \quad x_3 = [6 \ 5], \quad x_4 = [8 \ 9]$$

Mean vector

$$\mu = \frac{1}{4} \sum_{i=1}^4 x_i = \frac{1}{4} [2 + 4 + 6 + 8 \ 3 + 7 + 5 + 9] = [5 \ 6]$$

$$\tilde{x}_1 = [-3 \ -3], \quad \tilde{x}_2 = [-1 \ 1], \quad \tilde{x}_3 = [1 \ -1], \quad \tilde{x}_4 = [3 \ 3]$$

Using $1/n$ formula ($n = 4$):

$$\text{cov}(X_1, X_2) = \frac{1}{4} \sum (X_{1i} - X_1^-)(X_{2i} - X_2^-)$$

$$\text{var}(X_1) = \frac{1}{4} [(-3)^2 + (-1)^2 + 1^2 + 3^2] = \frac{20}{4} = 5$$

$$\text{var}(X_2) = \frac{1}{4} [(-3)^2 + 1^2 + (-1)^2 + 3^2] = \frac{20}{4} = 5$$

$$\text{cov}(X_1, X_2) = \frac{1}{4} \sum (X_{1i} - X_1^-)(X_{2i} - X_2^-) = \frac{1}{4} [(-3)(-3) + (-1)(1) + (1)(-1) + (3)(3)] = \frac{9-1-1+9}{4}$$

Covariance matrix:

$$\Sigma = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$$

Interpretation:

Off diagonal entry $\text{cov}(X_1, X_2) = 4 > 0$ hence both variables tend to increase together (positive linear relationship).

Diagonal entries $\text{var}(X_1) = \text{var}(X_2) = 5$ shows they have equal sample spread along both feature dimensions.

C.

$$a = \begin{bmatrix} 8 \\ 9 \end{bmatrix}, \quad b = \begin{bmatrix} 8 \\ 3 \end{bmatrix}$$

$$\text{mean } \mu = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$$a - \mu = \begin{bmatrix} 3 \\ 3 \end{bmatrix}, \quad b - \mu = \begin{bmatrix} 3 \\ -3 \end{bmatrix}$$

Inverse covariance matrix:

$$\det(\Sigma) = (5)(5) - (4)(4) = 25 - 16 = 9$$

$$\Sigma^{-1} = \frac{1}{\det(\Sigma)} \begin{bmatrix} 5 & -4 & -4 & 5 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 5 & -4 & -4 & 5 \end{bmatrix}$$

$$d_M^2(x, \mu) = (x - \mu)^T \Sigma^{-1} (x - \mu):$$

$$d_M^2(a, \mu) = \begin{bmatrix} 3 & 3 \end{bmatrix} \left(\frac{1}{9} \begin{bmatrix} 5 & -4 & -4 & 5 \end{bmatrix} \right) \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 3 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \frac{18}{9} = 2$$

$$d_M^2(b, \mu) = \begin{bmatrix} 3 & -3 \end{bmatrix} \left(\frac{1}{9} \begin{bmatrix} 5 & -4 & -4 & 5 \end{bmatrix} \right) \begin{bmatrix} 3 \\ -3 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 3 & -3 \end{bmatrix} \begin{bmatrix} 27 \\ -27 \end{bmatrix} = \frac{162}{9} = 18$$

Point b is significantly less plausible under fitted gaussian because

$d_M^2(b, \mu) = 18 \gg d_M^2(a, \mu) = 2$. Point a moves in same direction as positive covariance trend (both features higher), while point b violates this covariance structure.

D.

Off diagonal value = 4 > 0. Since covariance is positive, X_1 and X_2 increase together, so gaussian density contours are tilted upwards along $x_2 = x_1$ line.

A3

A.

Given $y_i = w^T x_i + \epsilon_i, \epsilon_i \sim N(0, \sigma^2)$

$$E[\epsilon_i] = 0$$

$$E[y_i] = w^T x_i + 0, \quad \text{var}(y_i) = 0 + \sigma^2$$

$$p(y_i | x_i, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - w^T x_i)^2}{2\sigma^2}\right)$$

Since noise terms ϵ_i are independent, we can simply product all of them

$$p(y|X, w) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - w^T x_i)^2}{2\sigma^2}\right)$$

Taking log likelihood:

$$\log p(y|X, w) = \sum_{i=1}^n \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(y_i - w^T x_i)^2}{2\sigma^2} \right] = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2$$

To maximize

$$\argmax_w \log p(y|X, w) = \argmax_w \left[-\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|Xw - y\|_2^2 \right]$$

$-\frac{n}{2} \log(2\pi\sigma^2)$ is constant wrt w so it can be dropped. Scaling factor $\frac{1}{2\sigma^2}$ also doesn't change argmin.

Flipping sign (maximizing positive $\rightarrow$ minimizing negative):

$$\argmax_w \log p(y|X, w) = \argmin_w \sum (y_i - w^T x_i)^2 = \argmin_w \|Xw - y\|_2^2$$

Hence maximizing likelihood is exactly equivalent to minimizing sum of squared errors.

B.

$$w \sim N(0, \tau^2 I)$$

We want to find w that maximizes posterior $p(w|X, y)$.

$$\text{By Bayes rule: } p(w|X, y) = \frac{p(y|X, w)p(w)}{p(y|X)}$$

We can drop evidence $p(y|X)$ as it doesn't depend on w .

$$p(w|X, y) \propto p(y|X, w)p(w)$$

$$\log p(w|X, y) = \log p(y|X, w) + \log p(w) + C$$

$$w_{MAP} = \argmax_w [\log p(y|X, w) + \log p(w)]$$

$$\text{Likelihood term: } \log p(y|X, w) = -\frac{1}{2\sigma^2} \|Xw - y\|_2^2 + C$$

$$\text{Gaussian prior } w \sim N(0, \tau^2 I) \Rightarrow p(w) \propto \exp\left(-\frac{1}{2\tau^2} \|w\|_2^2\right) \Rightarrow \log p(w) = -\frac{1}{2\tau^2} \|w\|_2^2 + C$$

Substituting into MAP and converting to minimization:

$$w_{MAP} = \underset{w}{\operatorname{argmin}} \left[\frac{1}{2\sigma^2} \|Xw - y\|_2^2 + \frac{1}{2\tau^2} \|w\|_2^2 \right]$$

Multiply whole expression by $2\sigma^2$ (constant, doesn't change argmin):

$$w_{MAP} = \underset{w}{\operatorname{argmin}} \left[\|Xw - y\|_2^2 + \frac{\sigma^2}{\tau^2} \|w\|_2^2 \right]$$

This gives exactly Ridge regression formula if we set regularization parameter $\lambda = \frac{\sigma^2}{\tau^2}$.

C.

If we use independent Laplace prior on weights $p(w_j) = \frac{\alpha}{2} \exp(-\alpha|w_j|)$:

$$p(w) = \prod_{j=1}^d \frac{\alpha}{2} \exp(-\alpha|w_j|) \propto \exp\left(-\alpha \sum_{j=1}^d |w_j|\right) = \exp(-\alpha \|w\|_1)$$

Taking log prior: $\log p(w) = -\alpha \|w\|_1 + C$

Combining with log likelihood in MAP objective:

$$w_{MAP} = \underset{w}{\operatorname{argmax}} [\log p(y|X, w) + \log p(w)] = \underset{w}{\operatorname{argmin}} \left[\frac{1}{2\sigma^2} \|Xw - y\|_2^2 + \alpha \|w\|_1 \right]$$

Multiplying by $2\sigma^2$:

$$w_{MAP} = \underset{w}{\operatorname{argmin}} \left[\|Xw - y\|_2^2 + \lambda_1 \|w\|_1 \right] \text{ where } \lambda_1 = 2\sigma^2\alpha \text{ (Lasso formulation).}$$

Sparsity explanation:

The L_1 penalty $|w_j|$ has a sharp corner (non-differentiable point) at $w_j = 0$ with constant derivative magnitude $\pm \lambda_1$ regardless of how small w_j is. This constant force can push parameters to become exactly zero if unpenalized loss derivative is smaller than λ_1 . In contrast, L_2 Ridge penalty $\lambda_2 w_j^2$ has derivative $2\lambda_2 w_j$ which goes to zero as $w_j \rightarrow 0$, so it shrinks weights close to zero but rarely sets them to exact zero.

D.

For logistic model $p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$ where $\operatorname{logit} z_i = w^T x_i$.

Bernoulli likelihood for $y_i \in \{0, 1\}$: $p(y_i|x_i, w) = p_i^{y_i}(1 - p_i)^{1-y_i}$

Product of independent likelihoods gives log likelihood:

$$\log L(w) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Negating and taking average gives Binary Cross Entropy loss:

$$L(w) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Applying chain rule for derivative wrt w :

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p} \times \frac{\partial p}{\partial z} \times \frac{\partial z}{\partial w}$$

Computing intermediate terms:

$$1. \quad \frac{\partial L_i}{\partial p_i} = -\left(\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i}\right) = \frac{p_i - y_i}{p_i(1-p_i)}$$

$$2. \quad \frac{\partial p_i}{\partial z_i} = \frac{\partial \sigma(z_i)}{\partial z_i} = p_i(1 - p_i)$$

$$3. \quad \frac{\partial z_i}{\partial w} = x_i$$

Combining step 1 and 2:

$$\frac{\partial L_i}{\partial z_i} = \frac{\partial L_i}{\partial p_i} \times \frac{\partial p_i}{\partial z_i} = \frac{p_i - y_i}{p_i(1-p_i)} \times p_i(1 - p_i) = (p_i - y_i)$$

Multiplying by $\frac{\partial z_i}{\partial w} = x_i$ and averaging over all n data points:

$$\nabla_w L = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) x_i = \frac{1}{n} X^T (p - y)$$

B1

A.SPLITTING

Roll number: 2025390 $\Rightarrow$ seed $s = 5390$.

Using `np.random.default_rng(5390)` to permute 730 rows of `train.csv`.

Split sizes:

Fit set (70%): 510 rows

Validation set (15%): 109 rows

Holdout set (15%): 111 rows

First 5 row indices in each split:

Fit: [429, 467, 399, 682, 37]

Val: [534, 709, 360, 484, 404]

Holdout: [646, 542, 71, 334, 412]

B.OLS CLOSED FORM VS GD

Prepending a column of 1s for bias term on raw features.

Closed form solution $(X^T X) \hat{\theta} = X^T y$ using `np.linalg.solve`:

$\hat{\theta}_{CF} = [-66.70009, 0.12996, 0.11509, 0.08820, -0.08525, 0.41344, 0.06233, 0.06264, 0.10503, -0.05000]$

Fit MSE = 103.74451, Val MSE = 113.07048

Batch GD on raw features ($\eta = 10^{-7}$, $T = 20000$ iterations):

$\hat{\theta}_{GD} = [-0.00023, 0.06513, 0.09466, 0.07704, -0.22251, 0.04893, 0.00944, 0.01167, 0.10374, 0.05000]$

Fit MSE = 109.23333, Val MSE = 110.64180

Norm difference $\|\hat{\theta}_{GD} - \hat{\theta}_{CF}\|_2 = 67.48308$

Why they should approach same optimum:

there is only one point where gradient is zero the closed form solution gives that point directly however the gradient descent form approaches it if we do proper iteration and with proper learning rate it will eventually reach that optimum point

(c) Feature Scaling and Convergence

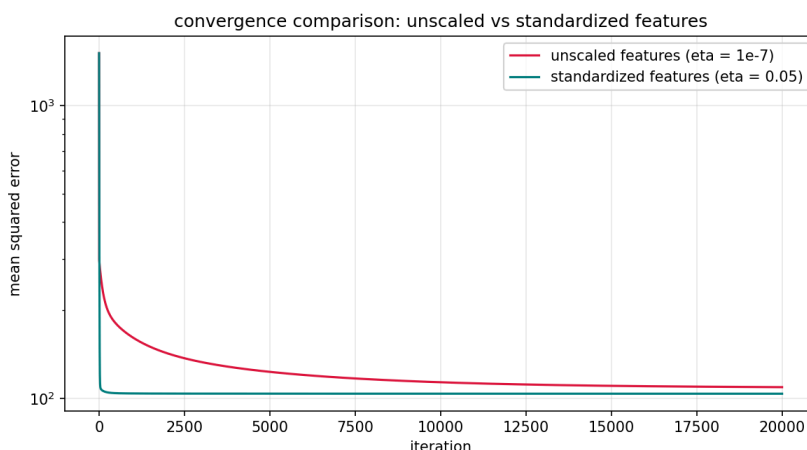
Standardizing features using the formula $z_j = (x_j - \mu_j)/\sigma_j$.

Then gradient descent with $\eta = 0.05$ and $T = 20000$:

Will give these fit mse and val mse

103.74601, 113.20079

(Standardized OLS closed form Fit MSE = 103.74451, Val MSE = 113.07048)



Explanation of convergence:

Initially when data is uninitialized they have different standard feature so it reaches the minimum very slowly Gradient descent with a very small learning rate (10^{-7}) so it converges very slowly. After standardizing features, the contours become circular and well conditioned, so we can take much bigger step size (0.05) and reach the minimum very quickly.

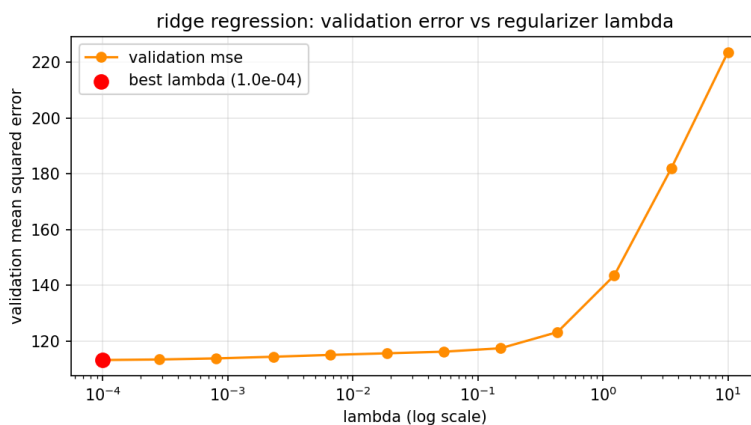
D. ridge from scratch

Using standardized features and unpenalized bias matrix $P = \text{diag}(0, 1, \dots, 1)$.

Formula: $\hat{\theta} = (X^T X + n\lambda P)^{-1} X^T y$ using `np.linalg.solve` and taking the lambda which have the lowest val

We check over 12 different values of lambda using `np.logspace`

λ	Fit MSE	Val MSE	Nonzero weights
1.0000×10^{-4}	103.74557	113.19402	13
2.8480×10^{-4}	103.75123	113.38700	13
8.1113×10^{-4}	103.77611	113.77834	13
2.3101×10^{-3}	103.84954	114.36688	13
6.5793×10^{-3}	104.04793	115.02613	13
1.8738×10^{-2}	104.63010	115.60494	13
5.3367×10^{-2}	106.14778	116.18598	13
1.5199×10^{-1}	109.76582	117.41297	13
4.3288×10^{-1}	120.03482	123.20046	13
1.2328×10^0	146.07418	143.52284	13
3.5112×10^0	189.83435	181.82487	13
1.0000×10^1	236.23370	223.68007	13



Selected Ridge $\lambda = 1.0000 \times 10^{-4}$ with validation MSE = 113.19402.

(e) lasso

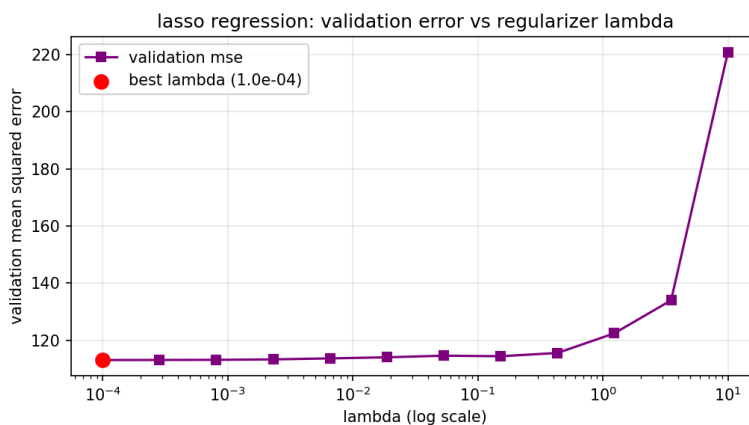
$$L = \frac{2}{n} \lambda_{\max}(X^T X) = 6.63412 \Rightarrow \eta = 0.9/L = 0.13566. (\text{given in the question})$$

$v = \theta - \eta g(\theta)$, keep bias v_0 unpenalized, soft threshold remaining:

$$\theta_j = \text{sign}(v_j) \max(|v_j| - \eta \lambda, 0).$$

results

λ	Fit MSE	Val MSE	Nonzero	Tol reached	Iterations
1.0000×1	103.74452	113.08266	13	False	20000
2.8480×1	103.74456	113.10063	13	False	20000
8.1113×1	103.74487	113.15203	13	False	20000
2.3101×1	103.74735	113.30007	13	False	20000
6.5793×1	103.76189	113.65705	12	True	7173
1.8738×1	103.79410	114.03258	12	True	7098
5.3367×1	103.94441	114.60257	11	True	4951
1.5199×1	104.48266	114.40699	11	True	4539
4.3288×1	106.14649	115.54890	10	True	2980
1.2328×1	111.00169	122.42134	9	True	2590
3.5112×1	128.48168	133.98428	6	True	1629
1.0000×1	220.17902	220.89407	4	True	566



Selected Lasso $\lambda = 1.0000 \times 10^{-4}$ with validation MSE = 113.08266.

F.model selection and kaggle

Model	Selected λ	Fit MSE	Val MSE	Nonzero weights
OLS	–	103.74451	113.07048	13
Ridge	$1.0000 \times 10^{-}$	103.74557	113.19402	13
Lasso	$1.0000 \times 10^{-}$	103.74452	113.08266	13

Chosen model: OLS has the lowest validation MSE (113.07048).

Holdout MSE on unseen test set: 130.41318.

Refitted OLS on all 730 training samples:

$$\hat{\theta}_{refit} = [35.66459, 20.02183, 9.44507, 5.84861, -1.94788, 2.04940, 6.99397, 7.53331, 7.20887, -9.44507]$$

Kaggle RMSE Score: 10.29350

(g) B1 Discussion

1. Without regularization, the model fits noise in the training set, causing high validation error (overfitting). Increasing the penalty shrinks the weights, which intentionally worsens the training set fit and increases training MSE. However, this weight reduction prevents the model from overreacting to noise, allowing it to generalize better and lower the validation MSE.
2. Ridge has the form $\lambda^2 w$ so by taking derivative it becomes $2\lambda w$ so it approaches to 0 when $w \rightarrow 0$ without making them exact 0 but lasso on the other hand has only λ so it might become exact zero removing some features
3. For correlated features Ridge shares the weight equally between both features. Lasso tends to select one of them and set the weight of other correlated feature to 0. As it has only λ factor instead of λ^2
4. OLS has the lowest fit mse as well as validation mse. However, holdout MSE (130.41) is higher than fit MSE (103.74), so training error is obviously better than error on unseen data

B2

Data generated with seed $10000 + 5390 = 15390$ ($n = 600$), split using seed $20000 + 5390 = 25390$ into fit (420), val (90), and holdout (90). Standardized with fit statistics.

First 5 indices in splits:

Fit: [542, 559, 317, 42, 180]

Val: [449, 280, 183, 222, 119]

Holdout: [32, 422, 246, 597, 557]

A. Numerically stable loss and gradient

Sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$ for $z \geq 0$, and $\frac{e^z}{1+e^z}$ for $z < 0$.

Stable BCE loss: $L(\theta) = \frac{1}{n} \sum [\text{logaddexp}(0, z_i) - y_i z_i]$

Gradient: $\nabla L(\theta) = \frac{1}{n} X^T (p - y)$

Why logaddexp is safer:

If we calculate probabilities p first and then take $\log p$, wehn $p=0$ or extreama value of z resulting in $\log(0) = -\infty$. Using $\text{logaddexp}(0, z) = \log(1 + e^z)$ we can avoid this underflow and overflow

B. Finite-difference gradient checker

Evaluation point θ_{check} sampled with seed $30000 + 5390 = 35390$, $\epsilon = 10^{-5}$:

$\theta_{check} = [-0.01138, -0.05959, 0.21545, 0.19693]$

Analytical gradient: $[0.0614860, -0.2072442, 0.2432978, -0.0223923]$

Numerical gradient: $[0.0614860, -0.2072442, 0.2432978, -0.0223923]$

Max absolute error = $4.9342 \times 10^{-12} < 10^{-5} \Rightarrow \text{PASS}$.

c. Learning rate and diagnosis

Trained unregularized logistic regression for $T = 5000$ iterations:

η	Fit BCE	Val BCE	Observed behavior
0.01	0.49221	0.36096	Slow steady decrease, hasn't fully converged
0.10	0.49218	0.35951	Smooth fast convergence
1.00	0.49218	0.35951	Fast smooth convergence without oscillations



Selected learning rate based on validation loss: $\eta = 1.0$.

D.L2 REGULARIZATION AND MAP

Regularized gradient: $\nabla L = \frac{1}{n}X^T(p - y) + \lambda P\theta$ with $P = \text{diag}(0, 1, 1, 1)$.

λ	Val BCE	$\ w\ _2$	Learned parameters θ
0.0	0.35951	1.73106	$[-0.3958, 1.1140]$
10^{-3}	0.36063	1.70832	$[-0.3932, 1.1010]$
10^{-2}	0.37021	1.54067	$[-0.3737, 1.0000]$
10^{-1}	0.43659	0.90364	$[-0.3064, 0.6040]$

Selected $\lambda = 0.0$ (Val BCE = 0.35951).

MAP connection:

As λ increases, the weight norm $\|w\|_2$ decreases from 1.73 to 0.90. From part A3(b),

L2 penalty corresponds to placing a zero mean Gaussian prior on weights $w \sim N(0, \tau^2 I)$ where $\lambda = 1/\tau^2$. Increasing λ is like decreasing prior variance τ^2 , which forces the weights closer to 0.

E. HOLD OUT DECISION

Evaluating selected model on holdout set ($n = 90$):

Holdout BCE = 0.38512

Holdout Accuracy (threshold 0.5) = 81.11% (73/90)

Confusion Matrix [[TN, FP], [FN, TP]]:

[48 8 9 25]

Threshold comparison:

Threshold t	False Positives (FP)	False Negatives (FN)	Accuracy
0.30	18	2	77.78%
0.50	8	9	81.11%
0.70	2	19	76.67%

Explanation:

Changing threshold changes the classification cutoff rule (predicting 1 if $p \geq t$), which trades off between false alarms and misses. But it does not retrain or change the underlying model weights θ or the predicted probabilities $P(y = 1|x)$.

F. FAILURE ANALYSIS

Holdout sample with maximum individual BCE penalty:

Sample index in holdout: 52

Standardized features: [0.82971, - 0.42506, 0.27683]

True label: 0

Predicted probability: 0.76748

Predicted class (threshold 0.5): 1

Individual BCE loss = 1.45876

Discussion:

This sample gets a high BCE penalty because the model was very confident that it belonged to class 1 ($p \approx 0.77$) while the true label was actually 0, so the $-\log(1 - p)$ loss becomes large. Having 100% training accuracy does not mean 100% future accuracy because training data can have noise and overfitting on training samples leads to misclassifications on unseen test data.